

Bài tập thực hành tuần 7

Họ và tên: Nguyễn Văn Phúc Huy
MSSV: 23110163
Lớp: 23TTH (Sáng thứ 6)

```
In [3]: import math
import matplotlib.pyplot as plt
import numpy as np
import time
import random
import heapq
import sys
from time import perf_counter
from math import log2
```

Bài tập:

Câu hỏi:

Bài 1. Một mảng A có n phần tử. Ta gọi $x \in A$ là phần tử "lần chiếm cấp độ 10" nếu như x xuất hiện ít nhất $\left\lceil \frac{n}{10} \right\rceil$ lần trong A .

1. Xây dựng một thuật toán tìm phần tử lần chiếm cấp 10 trong một mảng.
2. Liệu có tồn tại thuật toán có độ phức tạp $O(n)$ để tìm ra phần tử kiểu này không?

```
In [4]: def generate_a(n):
    return [random.randint(0, 500) for _ in range(n)]

def find_div10_element(a):
    threshold = math.ceil(len(a) / 10)
    count = {}

    for x in a:
        if x in count:
            count[x] += 1
        else:
            count[x] = 1

    for x in count:
        if count[x] >= threshold:
            return x

    return None

n_test = 100
print(f'n = {n_test}')
a_test = generate_a(n_test)

element = find_div10_element(a_test)
if element:
    print(f' -> Tồn tại phần tử lần chiếm cấp độ 10 là {element}\n')
else:
    print(f' -> Không tồn tại phần tử lần chiếm cấp độ 10\n')

# Caculate average time and plot
n_values = list(range(10000, 500001, 20000))
avg_times = []
num_trials = 20

for n in n_values:
    total_time = 0
```

```

for _ in range(num_trials):
    a = generate_a(n)

    start_time = time.perf_counter()
    find_div10_element(a)
    end_time = time.perf_counter()

    total_time += (end_time - start_time)

avg_times.append(total_time / num_trials)

C = avg_times[-1] / n_values[-1]
bigO_line = [C * n for n in n_values]

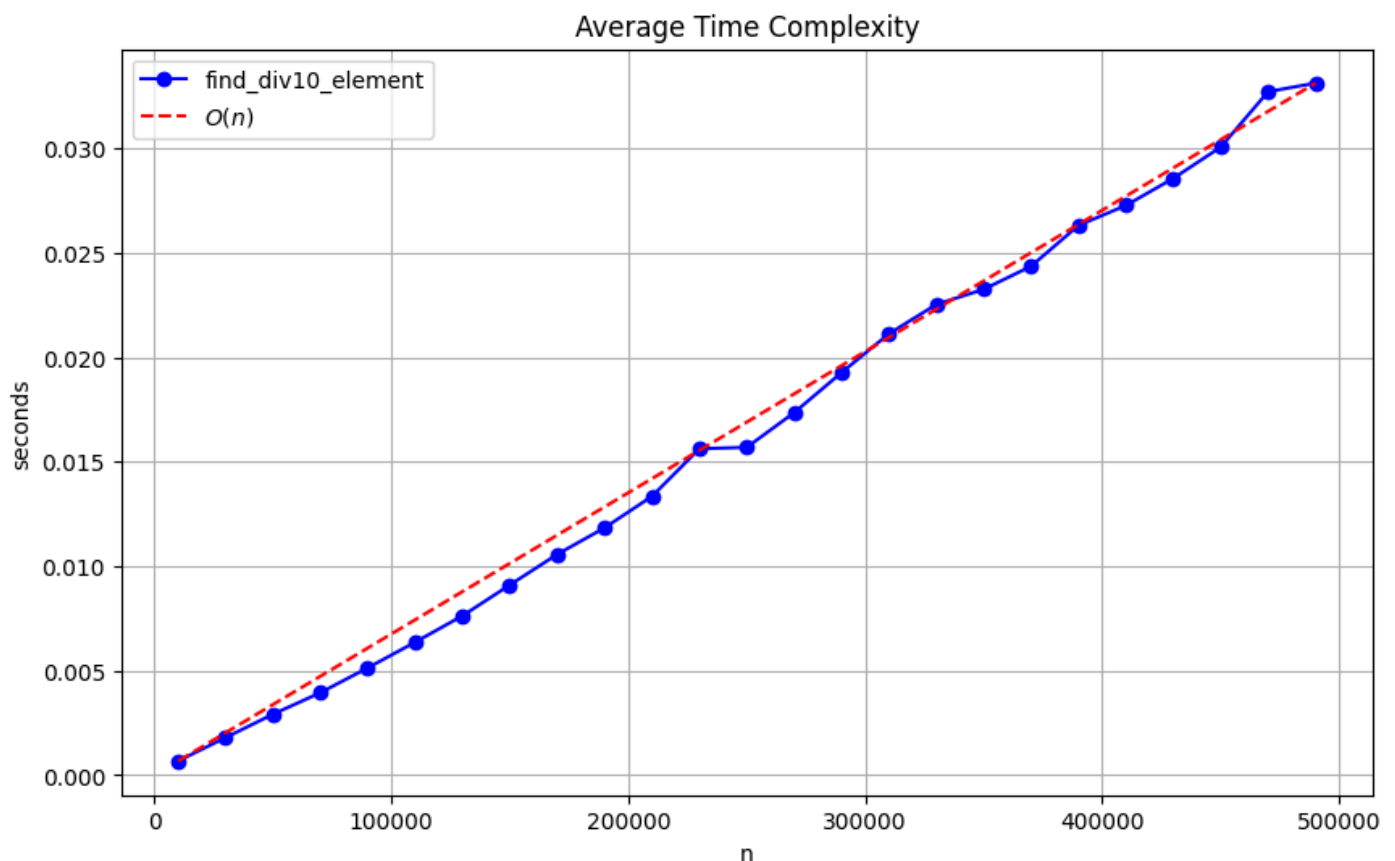
plt.figure(figsize=(10, 6))
plt.plot(n_values, avg_times, marker='o', color='blue', label='find_div10_element')
plt.plot(n_values, bigO_line, linestyle='--', color='red', label=r'$O(n)$')

plt.title('Average Time Complexity')
plt.xlabel('n')
plt.ylabel('seconds')
plt.legend()
plt.grid(True)
plt.show()

```

n = 100

-> Không tồn tại phần tử lần chiếm cấp độ 10



Hàm `generate_a(n)` tạo ra một danh sách gồm n số nguyên ngẫu nhiên trong khoảng từ 0 đến 500.

```

def generate_a(n):
    return [random.randint(0, 500) for _ in range(n)]

```

Hàm `find_div10_element(a)` thực hiện việc tìm kiếm phần tử lần chiếm cấp độ 10 trong mảng a. Thuật toán này hoạt động như sau:

1. Tính ngưỡng `threshold` bằng cách lấy chiều dài của mảng a chia cho 10 và làm tròn lên.
2. Sử dụng một từ điển `count` để đếm số lần xuất hiện của mỗi phần tử trong mảng a.
3. Duyệt qua mảng a và cập nhật số lần xuất hiện của mỗi phần tử trong từ điển `count`.
4. Sau khi đếm xong, duyệt qua từ điển `count` để kiểm tra xem có phần tử nào có số lần xuất hiện lớn hơn hoặc bằng ngưỡng `threshold` hay không. Nếu có, trả về phần tử đó. Nếu không có phần tử nào thỏa mãn, trả về `None`.

```

def find_div10_element(a):
    threshold = math.ceil(len(a) / 10)

```

```

count = {}

for x in a:
    if x in count:
        count[x] += 1
    else:
        count[x] = 1

for x in count:
    if count[x] >= threshold:
        return x

return None

```

Chương trình chính thực hiện các bước sau:

1. Định nghĩa giá trị `n_test` để kiểm tra thuật toán với một mảng có 100 phần tử.
2. Tạo một mảng `a_test` bằng cách gọi hàm `generate_a(n_test)`.
3. Gọi hàm `find_div10_element(a_test)` để tìm phần tử lần chiếm cấp độ 10 và in kết quả ra màn hình.

```

n_test = 100
print(f'n = {n_test}')
a_test = generate_a(n_test)
element = find_div10_element(a_test)
if element:
    print(f' -> Tồn tại phần tử lần chiếm cấp độ 10 là {element}\n')
else:
    print(f' -> Không tồn tại phần tử lần chiếm cấp độ 10\n')

```

Extra: Tính toán thời gian trung bình và vẽ đồ thị Sau khi tính giá trị trung bình của thời gian thực thi cho các giá trị n khác nhau, chúng ta sẽ vẽ đồ thị để so sánh thời gian thực thi của hàm `find_div10_element` với đường $O(n)$ để minh họa độ phức tạp thời gian của thuật toán.

```

n_values = list(range(10000, 500001, 20000))
avg_times = []
num_trials = 20

for n in n_values:
    total_time = 0
    for _ in range(num_trials):
        a = generate_a(n)

        start_time = time.perf_counter()
        find_div10_element(a)
        end_time = time.perf_counter()

        total_time += (end_time - start_time)

    avg_times.append(total_time / num_trials)

C = avg_times[-1] / n_values[-1]
bigO_line = [C * n for n in n_values]

```

```

plt.figure(figsize=(10, 6))
plt.plot(n_values, avg_times, marker='o', color='blue', label='find_div10_element')
plt.plot(n_values, bigO_line, linestyle='--', color='red', label=r'$O(n)$')

plt.title('Average Time Complexity')
plt.xlabel('n')
plt.ylabel('seconds')
plt.legend()
plt.grid(True)
plt.show()

```

Kết quả sẽ là một đồ thị hiển thị thời gian trung bình của hàm `find_div10_element` so với đường $O(n)$, giúp minh họa rằng thuật toán có độ phức tạp thời gian là $O(n)$.

Bài tập:

Câu hỏi:

Bài 2. Cho mảng A và B đã được sắp xếp theo thứ tự tăng dần và có n phần tử. Hãy xây dựng 1 thuật toán khả thi để merge hai mảng A và B thành mảng có $2n$ phần tử đã được sắp xếp theo thứ tự tăng dần.

```
In [5]: def generate_a_sorted(n):
        return sorted([random.randint(1, 500) for _ in range(n)])

def merge(a, b):
    n = len(a)
    C = []
    i, j = 0, 0

    while i < n and j < n:
        if a[i] <= b[j]:
            C.append(a[i])
            i += 1
        else:
            C.append(b[j])
            j += 1

    while i < n:
        C.append(a[i])
        i += 1

    while j < n:
        C.append(b[j])
        j += 1

    return C

n_test = 5
a_test = generate_a_sorted(n_test)
b_test = generate_a_sorted(n_test)
print(f'a = {a_test}')
print(f'b = {b_test}')
print(f'Merged = {merge(a_test, b_test)}\n')

n_values = list(range(10000, 300001, 20000))
avg_times = []
num_trials = 15

for n in n_values:
    total_time = 0
    for _ in range(num_trials):
        a = generate_a_sorted(n)
        b = generate_a_sorted(n)

        start_time = time.perf_counter()
        merge(a, b)
        end_time = time.perf_counter()

        total_time += (end_time - start_time)

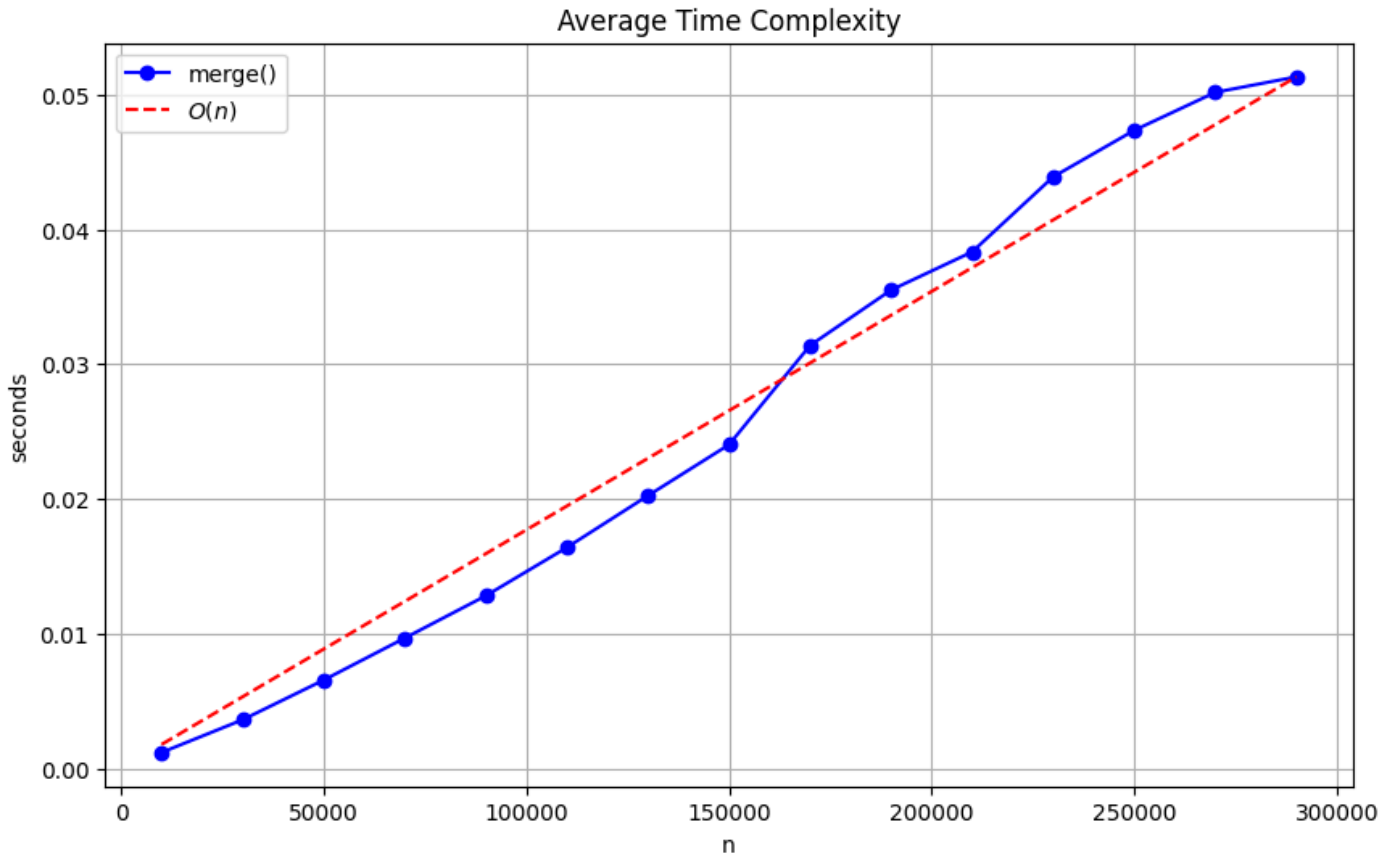
    avg_times.append(total_time / num_trials)

C = avg_times[-1] / n_values[-1]
bigO_line = [C * n for n in n_values]

# Vẽ đồ thị
plt.figure(figsize=(10, 6))
plt.plot(n_values, avg_times, marker='o', color='blue', label='merge()')
plt.plot(n_values, bigO_line, linestyle='--', color='red', label=r'$O(n)$')

plt.title('Average Time Complexity')
plt.xlabel('n')
plt.ylabel('seconds')
plt.legend()
plt.grid(True)
plt.show()
```

```
a = [85, 97, 227, 286, 399]
b = [50, 199, 208, 284, 286]
Merged = [50, 85, 97, 199, 208, 227, 284, 286, 286, 399]
```



Hàm `generate_a_sorted(n)` tạo ra một danh sách gồm n số nguyên ngẫu nhiên trong khoảng từ 1 đến 500 và sắp xếp chúng theo thứ tự tăng dần. Hàm `merge(a, b)` thực hiện việc gộp hai mảng đã được sắp xếp a và b thành một mảng mới C cũng được sắp xếp. Thuật toán này hoạt động như sau:

1. Khởi tạo một mảng rỗng C để lưu trữ kết quả gộp, và hai biến i, j để theo dõi vị trí hiện tại trong mảng a và b .
2. Sử dụng một vòng lặp để so sánh phần tử hiện tại của a và b . Nếu phần tử của a nhỏ hơn hoặc bằng phần tử của b , thêm phần tử đó vào C và tăng i lên 1. Ngược lại, thêm phần tử của b vào C và tăng j lên 1.
3. Sau khi một trong hai mảng đã được duyệt hết, sử dụng hai vòng lặp để thêm phần còn lại của mảng a hoặc b vào C .
4. Trả về mảng C đã được gộp và sắp xếp.

Chương trình chính thực hiện các bước sau:

1. Định nghĩa giá trị `n_test` để kiểm tra thuật toán với hai mảng có 5 phần tử.
2. Tạo hai mảng `a_test` và `b_test` bằng cách gọi hàm `generate_a_sorted(n_test)`.
3. Gọi hàm `merge(a_test, b_test)` để gộp hai mảng và in kết quả.
4. Tính toán thời gian trung bình của hàm `merge` cho các giá trị n khác nhau và vẽ đồ thị so sánh với đường $O(n)$ để minh họa độ phức tạp thời gian của thuật toán.

Extra: Tính toán thời gian trung bình và vẽ đồ thị Sau khi tính giá trị trung bình của thời gian thực thi cho các giá trị n khác nhau, chúng ta sẽ vẽ đồ thị để so sánh thời gian thực thi của hàm `merge` với đường $O(n)$ để minh họa độ phức tạp thời gian của thuật toán.

```
n_values = list(range(10000, 300001, 20000))
avg_times = []
num_trials = 15
for n in n_values:
    total_time = 0
    for _ in range(num_trials):
        a = generate_a_sorted(n)
        b = generate_a_sorted(n)

        start_time = time.perf_counter()
        merge(a, b)
        end_time = time.perf_counter()

        total_time += (end_time - start_time)
```

```

    avg_times.append(total_time / num_trials)
C = avg_times[-1] / n_values[-1]
bigO_line = [C * n for n in n_values]
plt.figure(figsize=(10, 6))
plt.plot(n_values, avg_times, marker='o', color='blue', label='merge() execution time')
plt.plot(n_values, bigO_line, linestyle='--', color='red', label=r'$O(n)$')
plt.title('Average Time Complexity')
plt.xlabel('n')
plt.ylabel('seconds')
plt.legend()
plt.grid(True)
plt.show()

```

Kết quả sẽ là một đồ thị hiển thị thời gian trung bình của hàm `merge` so với đường $O(n)$, giúp minh họa rằng thuật toán có độ phức tạp thời gian là $O(n)$.